

Git revert

The `git revert` command can be considered an 'undo' type command, however, it is not a traditional undo operation. Instead of removing the commit from the project history, it figures out how to invert the changes introduced by the commit and **appends a new commit with the resulting inverse content**. This prevents Git from losing history, which is important for the integrity of your revision history and for reliable collaboration.

Reverting should be used when you want to apply the inverse of a commit from your project history. This can be useful, for example, if you're tracking down a bug and find that it was introduced by a single commit. Instead of manually going in, fixing it, and committing a new snapshot, you can use `git revert` to automatically do all of this for you.

How it works

The `git revert` command is used for undoing changes to a repository's commit history. Other 'undo' commands like, `git checkout` and `git reset`, move the `HEAD` and branch ref pointers to a specified commit. `git revert` also takes a specified commit, however, `git revert` does not move ref pointers to this commit. A revert operation will take the specified commit, inverse the changes from that commit, and **create a new "revert commit"**. The ref pointers are then updated to point at the new **revert commit making it the tip of the branch**.

To demonstrate let's create an example repo using the command line examples below:

```
1  $ mkdir git_revert_test
2  $ cd git_revert_test/
3  $ git init .
4  Initialized empty Git repository in /git_revert_test/.git/
5  $ touch demo_file
6  $ git add demo_file
7  $ git commit -am"initial commit"
8  [main (root-commit) 299b15f] initial commit
9    1 file changed, 0 insertions(+), 0 deletions(-)
10   create mode 100644 demo_file
11  $ echo "initial content" >> demo_file
12  $ git commit -am"add new content to demo file"
13  [main 3602d88] add new content to demo file
14    n 1 file changed, 1 insertion(+)
15  $ echo "prepended line content" >> demo_file
16  $ git commit -am"prepend content to demo file"
17  [main 86bb32e] prepend content to demo file
18    1 file changed, 1 insertion(+)
19  $ git log --oneline
20  86bb32e prepend content to demo file
21  3602d88 add new content to demo file
22  299b15f initial commit
```

Here we have initialized a repo in a newly created directory named `git_revert_test`. We have made 3 commits to the repo in which we have added a file `demo_file` and modified its content twice. At the end of the repo setup procedure, we invoke `git log` to display the commit history, showing a total of 3 commits. With the repo in this state, we are ready to initiate a `git revert`.

```
1 $ git revert HEAD
2 [main b9cd081] Revert "prepend content to demo file" 1 file changed, 1 deletion(-)
```

`Git revert` expects a commit ref was passed in and will not execute without one. Here we have passed in the `HEAD` ref. This will revert the latest commit. This is the same behavior as if we reverted to commit `3602d8815dbfa78cd37cd4d189552764b5e96c58`. Similar to a merge, a revert will create a new commit which will open up the configured system editor prompting for a new commit message. Once a commit message has been entered and saved Git will resume operation. We can now examine the state of the repo using `git log` and see that there is a new commit added to the previous log:

```
1 $ git log --oneline 1061e79 Revert "prepend content to demo file" 86bb32e prepend content to
```

Note that the 3rd commit is still in the project history after the revert. Instead of deleting it, `git revert` added a new commit to undo its changes. As a result, the 2nd and 4th commits represent the exact same code base and the 3rd commit is still in our history just in case we want to go back to it down the road.

Common options

```
1 -e
2 --edit
```

This is a default option and doesn't need to be specified. This option will open the configured system editor and prompts you to edit the commit message prior to committing the revert

```
1 --no-edit
```

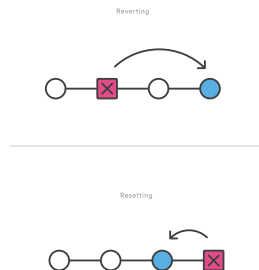
This is the inverse of the `-e` option. The revert will not open the editor.

```
1 -n
2 --no-commit
```

Passing this option will prevent `git revert` from creating a new commit that inverses the target commit. Instead of creating the new commit this option will add the inverse changes to the **Staging Index and Working Directory**. These are the other trees Git uses to manage the state of the repository. For more info visit the [git reset](#) page.

Resetting vs. reverting

It's important to understand that `git revert` undoes a single commit—it does not "revert" back to the previous state of a project by removing all subsequent commits. In Git, this is actually called a `reset`, not a `revert`.



Reverting has two important advantages over resetting. First, it doesn't change the project history, which makes it a "safe" operation for commits that have already been published to a shared repository. For details about why altering shared history is dangerous, please see the [git reset](#) page.

Second, `git revert` is able to target an individual commit at an arbitrary point in the history, whereas `git reset` can only work backward from the current commit. For example, if you wanted to undo an old commit with `git reset`, you would have to remove all of the commits that occurred after the target commit, remove it, then re-commit all of the subsequent commits. Needless to say, this is not an elegant undo solution. For a more detailed discussion on the differences between `git revert` and other 'undo' commands see [Resetting, Checking Out and Reverting](#).

Summary

The `git revert` command is a forward-moving undo operation that offers a safe method of undoing changes. Instead of deleting or orphaning commits in the commit history, a revert will create a new commit that inverses the changes specified. `Git revert` is a safer alternative to `git reset` in regards to losing work. To demonstrate the effects of `git revert` we leveraged other commands that have more in-depth documentation on their individual pages: [git log](#), [git commit](#), and [git reset](#).